ФАЙЛЫ И YAML-ЗАГОЛОВОК

kind	суффикс	подключение	после заголовка
`document`	—	собирается в ODT	разметка
`fragment`	.f	`|include`	разметка
`module`	.m	`|import`	только определения
`styles`	.s	`styles:` проекта/документа	код (Python)
`project`	.p	`project.p.prml`	код (Python)

Вид: kind → суффикс → документ. UTF-8, LF, NFC. Заголовок — YAML, без выражений.

```
---
kind: document
primula: {version: "1.1", stdlib: "1.1"}
doc: {title: "Договір", language: uk-UA}
params: {vat: true}       # |(vat)
references: [contract]    # документы проекта для |@
---
```

Ключи: `kind primula doc params structure styles references`; в `doc`: `title subject description creator organization language identifier revision status created modified keywords`.

Имя секции = имя в тексте и коде: `doc:` → `|(doc.title)`. В проекте — `doc_defaults`, `ref_documents`; прежние `document`/`documents` — ошибка P1120.

ЧТО ИДЁТ ПОСЛЕ |

`||`	литерал `|`
`(`	выражение `|(…)`
буква	длинная команда `|имя` (точка — модуль)
пунктуация, символ	короткая форма (справа)
пробел, конец строки, цифра	ошибка; также `|$ |[ |]`, `|–` (U+2013) вместо `|-`

ДЛИННАЯ ФОРМА КОМАНДЫ

```
|имя(арг, k=v){тело}      в строке
|имя(арг, k=v): текст     строчная (первой в строке)
|имя(арг):                окружение
  …
|end имя
```

- пробелов между именем, `(` и `{` нет; аргументы — выражения ядра, можно в несколько строк
- после команды идёт буква → разделитель: `|seller|;ом`; буквальное `:` — `|seller|::`
- тело `{…}` считается по балансу скобок; одиночная скобка — `|{ |}`
- тело макроса — параметр `body` (`|(body)`); динамический вызов: `|(fn(x))`, `|call(fn, x){…}`

ЗАКРЫТИЕ ОКРУЖЕНИЙ

`|end`	ближайшее окружение (не if/for/def), без проверки
`|end> |end>> |end.Имя |end!`	пункт уровня · стиль · блок `|!`, с проверкой
`|end имя`	любое именованное: `|end table`, `|end clause`, `|end style`, `|end admonition`
`|endif |endfor |enddef`	управляющие конструкции

КРАТКАЯ ↔ ПОЛНАЯ ФОРМА (РЕЗУЛЬТАТ ОДИНАКОВ)

краткая	полная
`|*{x} |/{x} |_{x}`	`|.Strong{x} |.Emphasis{x} |.Underline{x}`
`|^{x} |,{x} |-{x}`	`|.Superscript{x} |.Subscript{x} |.StrikeText{x}`
`|`{x} |?{x}`	`|.Code{x} |.Placeholder{x}`; `|end>` = `|end clause`
`|<<< Назва`	`|<(level=3) Назва`
`|> текст`	`|>: текст |end>` — явная область
`|!("note") текст`	`|!("note"): … |end!`
`|: a |& b`	`|row: |cell: a |end cell … |end row`
`|if(c){…} |for(x in xs){…}`	`|if(c):…|endif  |for(…):…|endfor`

КОРОТКИЕ ФОРМЫ — ПОЛНЫЙ ПЕРЕЧЕНЬ

`|| |{ |} |;`	литералы `| { }` · пустой разделитель (граница команды)
`|%`	комментарий до конца строки
`|(выр)`	значение выражения (как текст, не разметка)
`|\ |~`	разрыв строки · неразрывный пробел
`|*{} |/{} |_{}`	жирный · курсив · подчёркнутый = `|.Strong{} |.Emphasis{} |.Underline{}`
`|^{} |,{} |-{}`	верхний · нижний индекс · зачёркнутый
`|`{} |"{} |?{}`	моноширинный (`Code`) · кавычки по языку: uk, ru «…» „…“, en “…” ‘…’ · заполнитель черновика (`--strict` — ошибка; `placeholders("final")` — обычный текст)
`|.Имя{…} |.(выр){…}`	символьный стиль
`|.Имя: |.(выр):`	абзацный стиль, строка или окружение; закрытие `|end.Имя |end style`
`|@("id")`	перекрёстная ссылка, режим — вторым аргументом
`|link("url"){текст}`	внешняя гиперссылка; без тела — текст = адрес; стиль `structure_styles(link=)`
`|< … |<<<<<< |<(level=N)`	заголовок уровня 1–6; `style="Имя"` — абзацный стиль заголовка
`|> |>> …`	пункт, уровень = число `>`; закрытие `|end> |end clause`; `style=` — стиль пункта
`|- |# |-- |#-`	элемент списка: длина — уровень, последний знак — тип (`-` маркер, `#` номер)
`|!("вид")`	предупреждение/примечание; закрытие `|end! |end admonition`
`|: |& |=(выр)`	строка = `|row` · ячейка = `|cell` · отложенная формула ячейки; `|: |& Б` (с пробелом) — пустая первая ячейка

Любая короткая форма принимает `(аргументы)`; у `|> |- |#` параметра `level` нет. Структурные (`|< |> |- |# |! |:`) — только в начале строки.

КОД, КОММЕНТАРИИ, БУКВАЛЬНЫЙ ТЕКСТ

`|% текст`	комментарий строки
`|comment: … |end`	блочный комментарий (проверяется только баланс)
`|verbatim: … |end`	буквальный текст
`|verbatim(end="EOF"): … EOF`	свой маркер конца; сравнивается буквально
`|code: … |end`	Python-подмножество; в `.s`/`.p` — без обёртки, комментарии `#`

Отступ открывающей строки снимается со всего блока; иной отступ — ошибка.

АБЗАЦЫ И ТЕКСТ

- пустая строка — новый абзац; перевод строки внутри абзаца — пробел
- строчная форма (пункт, элемент списка, строка таблицы, заголовок) кончается в конце строки; многоабзацное — окружением
- в `|(…)` нельзя `None bool list record` (нужен `str()`, `join_list`); разметка из строки — только `markup(…)`
- дробные числа и даты — через `num`, `fmt_date`, иначе предупреждение

УСЛОВИЯ, ЦИКЛЫ, МАКРОСЫ, ПЕРЕМЕННЫЕ

`|if(n > 3){…}|elif(c){…}|else{…}`	в строке
`|for(x in xs){|(x) }`	в строке
`|if(c): … |elif(c): … |else: … |endif`	
`|for(it in items): … |endfor`	
`|def sign(name, role="директор"): …|(body)… |enddef`	макрос
`|let(rate=20, export=True, override=False)`	`|export(months=12)`, `|export("sign")`, `@export` над `def`
`|>(when=has("wifi")) …`	`when=` — на любой блоковой команде и строке таблицы

`has("x")` по `features: [x]`. `--review` включает все варианты с пометками. `|let` / `|code:` внутри `|if` верхнего уровня — ошибка.

ЗАГОЛОВКИ И ПУНКТЫ

форма	аргументы
`|< … |<<<<<<`	`id number level toc toc_title toc_level style when`; `|<(number=False)` — без номера; `level=` или число <, не оба сразу
`|> |>>`	`id continues block style toc toc_title toc_level when`; `|>(continues="id")` — продолжить нумерацию
`|>(…): … |end>`	пункт из нескольких абзацев (явная область)

Неявная область пункта кончается: абзацем, следующим `|>` того же/высшего уровня, `|<`, `|page section appendix scope document toc list_of header footer`. Списки, таблицы, `|!`, рисунки, пустые строки — внутри пункта.

СПИСКИ

```
|- маркер            |# номер
|-- 2-й уровень      |#- маркер под номером
|#(id="st", start=3) именованная; |#(continues="st")
|list("Стиль")       стиль ближайшего списка
|list("Стиль"): … |end list   (пустые строки внутри)
```

Нельзя смешивать `-` и `#` на одном уровне и пропускать уровень.

СТИЛИ, УМОЛЧАНИЯ, БЛОКИ

`|.Имя{…} |.Имя: …`	символьный / абзацный; `|.(выр)` — имя вычисляется
`|.Body(before="l", after="s", indent=2)`	параметры по месту: ступень шкалы spacing или длина
`|.(Indented("Clause",2)):`	производный стиль (`derive`)
`|set(paragraph=, list=, table=, heading=, clause=, …)`	стили конструкций до конца окружения; `list=None` — сброс
`|block("Имя"): … |end block`	стиль группы; или `block=` у `|> |! |table |figure |appendix`
`|fmt(color="#C00000", bold=True){…}`	прямое форматирование (предупреждение)

ССЫЛКИ |@

запись	результат
`|@("id")`	номер «1.2»
`|@("id","label")`	«п. 1.2», «розд. 2», «рис. 3», «табл. 1», «дод. А»
`|@("id","title")`	название / текст первого абзаца пункта
`|@("id","page")`	номер страницы (поле Writer)
`|@("id","label",case="loc")`	«розділі 2»; падежи `nom gen dat acc ins loc`

`id=` (допустимы `-` `.`; уникален) — у заголовка, пункта, рисунка, таблицы, приложения, сноски, раздела, `|!`; в сводном документе `act1:items`.

Другой документ проекта: документ с `references: [contract]` ищет ID, которого нет у него, в документах `ref_documents` проекта (текст — из реестра `*.refs.json`); ссылка — текст без гиперссылки, `page` — ошибка.

ПРИЛОЖЕНИЯ И ОБЛАСТИ НУМЕРАЦИИ

```
|appendix(id="a1", title="Специфікація",
  subtitle="до Договору", number=3 | letter="Б",
  page="Landscape", block="…"): … |end appendix
|scope("name", prefix="A"): … |end scope
```

Новая страница; «Додаток 1» (business) / «ДОДАТОК А» (report); свои счётчики: «Таблиця А.1». Имя `document` зарезервировано. Настройка — `appendices(designation=, letters=, title_format="{designation} {title}", toc=)`; приложение — заголовок структуры (навигатор Writer, закладки PDF).

ПОДКЛЮЧЕНИЕ ФАЙЛОВ

`|include("f.f.prml", part=, ns=, rename={}, level_shift=n, k=v)`	фрагмент; `path part key` известны при построении; `level_shift` 0–5 — сдвиг заголовков вниз; `ns`/`rename` — против конфликтов
`|include(key="k")`	`|part("x"):…|end part`
`|import("lib/b")`	модуль: `b.sign()` · без префикса `|import("lib/b","a","b")` · псевдоним `alias="x"`; `_имя` не экспортируется
`|document("c.prml", scope=, params={}, page=, level_shift=)`	часть сводного ODT; ссылка `|@("c:id")`

Путь — от файла, не выше корня проекта; `.prml` можно опустить. Ключи карты — `файл#часть` только для `|include`.

ТАБЛИЦА |TABLE(…): … |END TABLE

параметр	значение · по умолчанию
`cols`	список столбцов; по умолчанию равные (или `cols` стиля таблицы)
`style width align`	стиль · `"100%"` · `left center right`
`header footer`	строк шапки (1) · итогов (0)
`repeat_header keep_rows may_break`	все `True`
`frame rules`	рамка · `all rows cols none`
`caption id`	подпись с номером · цель ссылки
`block when live`	стиль блока · условие · `live=True` — формулы Writer
`data fields titles`	таблица из списка записей: `| table(data=items, fields=[…], titles=[…])`

Внутри допустимы только строки, `|caption:`, `|for|if`, `|code:`, `|include`, комментарии. Подпись с разметкой: `|caption: … |end caption`. Отступы до/после таблицы — `space_before`/`space_after` стиля (у `AdmonitionTable` 3 мм).

СТОЛБЦЫ COLS=[…]

`"40mm" "30%"`	фиксированная · доля таблицы
`"*" "2*"`	остаток, пропорционально весу
`"auto"`	по содержимому
`"20mm..60mm" "20mm.." "..60mm"`	в пределах / только нижняя / верхняя граница
`col(ширина, id=, align=, valign=, type=, currency=, digits=, paragraph=, min=, max=)`	`type="decimal|money|datetime"`; `currency="UAH"` → «грн»

СТРОКИ И ЯЧЕЙКИ

```
|: Назва |& К-сть |&(span=2, align="right") Разом
|:(keep=True, when=has("wifi")) Умовний рядок |& 1
|:|&(span=3) Разом       аргументы 1-й ячейки
|: |& Б                  пустая первая ячейка
|row:   |cell(rows=2, valign="top"): абзацы, списки,
  вложенные таблицы |end cell |end row     развёрнуто
```

ячейка	`span=N` гориз. · `rows=N` верт. · `align valign padding background paragraph border border_top border_bottom border_left border_right`
строка	`height min_height keep when` + свойства ячейки

Каскад: таблица → столбец → строка → ячейка. Ячеек в строке = столбцов (с учётом объединений); пустая — `|&`. Рамка: `"0.5pt solid #000000"` (`solid dashed dotted double none`).

ФОРМУЛЫ ЯЧЕЕК |=(…)

`this_row(c) cell(r,c)`	значение в текущей строке · ячейка (с 1; столбец — номер или id)
`column(c, rows=…) body_rows()`	значения столбца · строки между шапкой и итогами
`sum min max avg count`	агрегаты; `None` пропускается

```
|:|&(span=2, align="right") Разом
  |& |=(sum(column("sum", rows=body_rows())))
```

ПОДПИСИ, ОГЛАВЛЕНИЕ, СНОСКИ, ТЕРМИНЫ

```
|toc(max_level=3, title="ЗМІСТ", page_break=True)
|list_of("figure"|"table"|"abbr"|"index", title="…")
|footnote(id="fn"){текст}  |endnote{текст}  |@("fn")
|abbr("ПЗ","розшифровка"[, show=False][, text=])
|abbr("ПЗ")       |index("напруга", sub="живлення")
|toc_entry("Текст", level=1)   строка оглавления не из заголовка
```

В оглавление не входят пункты `|>`; `toc=True` включает пункт или более глубокий заголовок; `toc=False`, `toc_title=`, `toc_level=` — у `|<`, `|>`, `|appendix`.

ПРЕДУПРЕЖДЕНИЯ |!

```
|!("warning") Одна строка.
|!("danger", hazard="Висока напруга", consequence="…",
   symbol="assets/w012.svg", id=, block=, when=):
  …абзацы, списки…
|end!
```

Виды: `danger` НЕБЕЗПЕКА · `warning` ПОПЕРЕДЖЕННЯ · `caution` ОБЕРЕЖНО · `notice` УВАГА · `note` Примітка · `tip` Порада. `hazard` нужен у `danger` / `warning`.

СТРАНИЦЫ, РАЗРЫВЫ, КОЛОНТИТУЛЫ

```
|page("Appendix", restart=1, number_format="i")
|page("Landscape"): … |end page
|newpage      |newcolumn
|section(cols=2, gap="6mm", rule="0.5pt solid",
  balance=True, widths=["60%","40%"], id=):
  … |end section
|header("Portrait", which="all|first|left|right"):
  …разметка… |end header    (|footer — так же)
```

Поля: `|field.page |field.pages |field.chapter(level=1) |field.title`. `page_style(…)`: `parent size` (`A3 A4 A5 Letter` или `(w,h)`) `orientation margin` (верх, право, низ, лево) `cols col_gap col_rule header_gap header_min_height footer_gap footer_min_height number_format restart next background watermark mirror`.

ПАРАЛЛЕЛЬНЫЕ ПОТОКИ (§7.7)

```
|parallel(langs=["uk","en"], gap="8mm", rule="0.3pt solid",
  parity="structure|translation|none"):
|segment(id="s") |track("uk"){…} |track("en"){…}
|segment(id="t"):  |track("uk"): …  |track("en", rev=, status=): …
  |end segment     |segment(span="all"): … |end segment
|end parallel      track("note", width="30%", numbered=False,
  optional=True, primary=False, title="…", lang="en")
```

ID — `uk.id`; `|toc(track="en")`; `primula parallel F --stub|--stamp --track en [--segment s --status reviewed]|--check-status reviewed`.

ГРАФИКА

```
|img("logo.png", height="8mm")   |img(key="logo")
|img(svg=bar(30))   |figure(chart(d)): …   # SVG из кода
|img(svg=load_text("img/знак.svg"))         # SVG из файла
|figure("scheme.svg", width="120mm", id="fig-1"):
  Підпис рисунка |end figure
|frame(rel="page", x="120mm", y="25mm", width="70mm",
  wrap="through", layer="front"): … |end frame
|watermark("ПРОЄКТ", pages=["Portrait"], angle=45,
  color="#C8C8C8", size=None, opacity=50, font=None)
```

place	`inline left center right inside outside`; `valign top middle bottom`; `x y`
rel anchor	`para area page` · `char para page`
wrap layer	`none side both auto through contour` · `front back`
прочее	`gap width height rotate mirror crop opacity border padding style alt`

PNG, JPEG, SVG; пути от файла, внутри проекта. Одна из форм: путь, `key=`, `svg=` или значение `img(…)`. Свойства стиля фрейма — умолчания аргументов `|frame`.

СТАНДАРТНАЯ БИБЛИОТЕКА

числа	`num(x,digits=) percent round_to(x,step) ceil_to floor_to round_money`
деньги	`money(x,"UAH") money_words money_full(kop="digits|words", cap=, currency_style="short|full") vat(net,rate) with_vat vat_in without_vat split(total,[30,70])`
прописью	`words(n, gender="m|f|n", case=) ordinal date_words`
даты	`add_months add_years weeks hours minutes fmt_date(dt,"short|long|contract|{d} {month_gen}") fmt_time fmt_datetime weekday month_name workdays(d,n,calendar=) is_workday`
единицы	`qty(x,"V",tol=) qty_range(a,b,u) convert(x,"kW","W")`
текст	`plural(n,"день","дні","днів",frac=) join_list(xs,last=,sep=) cap translit sort_key nbsp_short`; весь текст — `typography(nbsp_short=True)`
данные	`group_by sum_by distinct`; `|import("std/math")`: `sqrt exp log log10 sin cos tan atan2 pi e floor ceil`
QR	`qr_svg(data, ecc=, size=, circle=) qr_matrix base64url(text, encoding=)`

Язык — `lang="uk\|ru\|en"`. `float` в денежных функциях — ошибка (нужен `dec()`). Ставка НДС передаётся явно.

ЧАСТЫЕ ОШИБКИ

- пробел, цифра, конец строки после `|`; `|–` (U+2013) вместо `|-`
- `|sellerом` читается как команда `sellerом` — нужен `|seller|;ом`
- `|end` не закрывает `|if|for|def` — нужны `|endif|endfor|enddef`; `|code:` — только строкой `|end`
- число `<` и `level=` вместе; пропуск уровня заголовка или списка
- в `when=` строка вместо логического значения; `|let` в `|if` верхнего уровня
- ключ `lines` и `primula: {version: "1.0"}` — ошибки (режим строк удалён)
- `document:/documents:` в заголовке — ошибка P1120 (теперь `doc`, `doc_defaults`, `ref_documents`)

ПРОЕКТ PROJECT.P.PRML

```
---
kind: project
name: "Договори 2026"    language: uk-UA
doc_defaults: {organization: "ТОВ «Постачальник»"}
styles: [styles/business.s.prml]
on_conflict: error    build_date: 2026-10-12
params: {city: "м. Харків"}    keys: {logo: a/l.svg}
ref_documents: {contract: c.prml}
extends: ../../project.p.prml
limits: {time: 20, recursion: 500, size: 10_000_000}
targets:
  - {source: c.prml, output: out/c.odt, pdf: true}
---
for row in load_csv("data/acts.csv"):
    build("act.prml", params=row,
          output=f"out/{row.no}.odt")
```

Приоритет параметров: цель → документ → проект. `batch_mode()` — только в коде проекта. Данные: `load_csv(путь, types={…}, sep=)` `load_yaml` `load_json` `load_ods` `load_xlsx` `load_text`. `extends:` — проект наследует базовый (параметры, ключи, `ref_documents`, `doc_defaults`, стили, код; цели не наследуются). `doc_defaults` — умолчания метаданных ODT; документ заменяет их поштучно.

СТИЛИ (МОДУЛЬ .S.PRML)

группы YAML	`paragraphs characters blocks tables lists frames pages` = функции `paragraph_style character_style block_style table_style list_style frame_style page_style`; наследование `parent`
абзац	`font size bold italic color background underline strike language align line_height space_before space_after left right first_line keep_with_next keep_together page_break_before widows orphans`
таблица	`width align frame rules border padding background header_* footer_* paragraph repeat_header keep_rows may_break space_before space_after cols`
список	`paragraph levels:{1:{type format prefix suffix bullet left hanging start letters}}`

Настройка: `structure_profile("business\|report")`, `structure_styles(link=, clause_number=, heading_number=, …)`, `numbering`, `captions`, `appendices`, `ref_labels`, `footnotes`, `quotes`, `abbreviations`, `admonition_words`, `placeholders("mark\|final")`, `typography(…)`, `derive(base, name=, **props)`. Слои: проект → `--style` → `styles` документа → код документа; `extends: std/styles/business` — основа модуля, слой перед ним. Наборы комплекта `std/styles/…`: `business letter technical report compact` (основа `default`).

ВЫЧИСЛИТЕЛЬНОЕ ЯДРО

- типы: `None bool decimal float str list tuple dict record datetime length content function`; целое — `decimal`; `2.20` → «2,20»; `1.5e3` — `float`; `round` — от нуля
- длины: `10*mm`, `"10mm"` (`mm cm pt inch`); даты: `date(2026,10,12)`, `dt + 5` — сутки
- есть: `def while for if`, генераторы, f-строки, `*args **kw`; **нет:** `class import from try except raise with global del lambda yield async await := match`
- функции: `len range enumerate zip sorted reversed min max sum any all abs round int float dec str bool list tuple dict record date datetime type getattr hasattr repr fail debug markup build_date`; методы `str list dict` как в Python
- верхний уровень: определения (ленивые, неизменяемые) и действия; лимиты `time` 20 с · `recursion` 500 · `size` 10^7
- отличия от Python подробно — на стр. 4

КОМАНДНАЯ СТРОКА

```
primula build файл [-o out.odt] [--pdf [f.pdf]] [--report r.json]
primula check файл [--dump-project | --dump-structure --dump-styles]
                   [--overlay ФАЙЛ=БУФЕР]
primula export файл --to md|html|docx [-o выход] [--pandoc-arg A]
primula commands [--format json] [--functions | --header]
primula console [файл] [--json]
```

`--project F` `--no-project` `--strict` `--review` `--build-date YYYY-MM-DD` `--style F` (повторяется) `--param ИМЯ=ЗНАЧ` (повторяется) `--format text|json` `--lang` `--timeout СЕК` `--unsafe` `--embed-sources` `--no-embed-report`. Код возврата: 0 успех, 2 ошибка. Диагностика `P0…P9`: CLI/ввод-вывод · синтаксис · имена/ядро · стили · структура/ссылки · таблицы · страницы/графика · модули/проект · ODF · техдок.

`--dump-*` и `commands` дают JSON для редакторов и Studio; `console` — проба кода и функций библиотеки (`:help имя`, `:functions`, `:vars`, `:reset`, `:context`, `:quit`).

ЧИСЛА: DECIMAL ВМЕСТО INT И FLOAT

Целое и число с точкой без экспоненты — `decimal`: точное, до 64 значащих цифр, помнит число знаков. Типа `int` нет: `type(5) == "decimal"`. `float` — только литерал с экспонентой `1.5e3` или `float(x)`; `decimal` с `float` → `float`.

Выражение	Primula	Python 3
`0.1 + 0.2 == 0.3`	`True`	`False`
`2.20 * 3`	`6.60`	`6.6000000000000005`
`7 / 2`	`3.5` decimal	`3.5` float
`1 / 3`	28 значащих цифр	16–17 цифр
`round(2.5)`, `round(-2.5)`	3, -3	2, -2
`round(2.675e0, 2)`	`2.68`	`2.67`
`-7 // 2`, `-7 % 2`	-4, 1	то же
`2 ** -1`	`0.5` decimal	`0.5` float
`10 ** 70`	P2309 (>64 цифр)	целое
`True + 1`	P2304	2
`[1, 2][2.0 - 1]`	2	TypeError

Округление одно — половина от нуля, `float` — по десятичной записи. Деньги — только `decimal` (`dec("0.20")`); денежные функции `float` не берут (P2703). В текст — `num money percent`, не `f"{x:,.2f}"` (формат Python английский).

СТРОКИ

- символы Unicode, NFC: `len("Київ") == 4`, срезы и итерация по символам
- методы — только: `split rsplit splitlines join strip lstrip rstrip replace startswith endswith find rfind count upper lower capitalize title isdigit isalpha isspace zfill format`; прочие (`center partition encode`) — P2313
- f-строки по синтаксису 3.11: внутри `{…}` нет кавычек того же вида (P1208) — `f"{d['k']}"`
- `"{0.name}".format(r)` — P2330; поле записи — `"{0[name]}"`; `"%s: %d" % (a, n)` можно
- `"x" * 2.5`, индекс `0.5` — P2322 (нужно целое значение)

СПИСКИ, КОРТЕЖИ, СЛОВАРИ

- генераторы списков и словарей, распаковка `a, *b = xs` — как в Python
- множеств нет: `{1, 2}` — P2101; уникальность — словарь или `not in`
- операторов `| & ^ << >> ~` нет; слияние словарей — `{**a, **b}`
- `is` / `is not` — только с `None` (P2103)
- ключ словаря — неизменяемое: `None bool decimal str datetime`, кортеж и запись из них; `float` и длина — P2317; `5` и `5.00` — один ключ
- `range()`, `reversed()` возвращают список; `key=` — обычный `def`, `lambda` нет
- методы списка: `append extend insert pop remove index count sort reverse`; словаря: `get keys values items update pop setdefault`
- `[1] + (2,)` — P2301, как TypeError в Python

ЗАПИСИ RECORD

```
r = record(назва="Папір", кількість=3)
r.назва   r["кількість"]   hasattr(r, "назва")
getattr(r, "ціна", 0)      r.replace(кількість=4)
dict(r)   record(**d)
```

Неизменяемая, **не последовательность**: `"x" in r`, `len(r)`, итерация — ошибки (у `namedtuple` `in` ищет среди значений!). Нет поля — P2314. Записи — параметры YAML, данные проекта, `load_csv/yaml/json`; YAML: `15.00` — decimal, `2026-10-12` — datetime. Словарь остаётся словарём: `d.a` — ошибка.

ЧЕГО НЕТ — ЧТО ВМЕСТО

`import`, `from`	`|import("std/math")`, модули `.m.prml`
`class`	`record(…)` и функции
`try except raise`	ошибка останавливает сборку; `fail("…")`, `getattr(…, default)`, `d.get`
`print open with`	`|(x)`, `debug(x)` — в отчёт; `load_*`
`lambda yield async`	`def`, функция, строящая список
`global nonlocal del`	новое имя
`for … else`, аннотации	флажок, `any()`; без типов
`x.__class__`	`type(x)` → строка `"decimal"`
часы, `random`	`build_date()` (`--build-date`); случайных чисел нет

ДАТЫ И ДЛИНЫ

Выражение	Primula (Python)
`date(2026,10,12) + 30`	`date("2026-11-11")` (TypeError, нужен timedelta)
`d2 - d1`	сутки, `decimal`, бывают дробные (timedelta)
`d + hours(36)`, `weeks(2)`	числа суток
`add_months(date(2026,1,31), 1)`	`2026-02-28`
`d.weekday`	атрибут 1–7, пн = 1 (метод 0–6)
`str(dt)`	`2026-10-12T14:30:00` (с `T`)
`float(d)`	сутки от 30.12.1899
`10*mm + 1*cm`	`length("20mm")` — единица левого
`72*pt == inch`, `(210*mm).cm`	`True` (допуск 0,001 мм), `21`

Один тип `datetime` без `import`, точность — секунда, поясов нет. Длины: `mm cm pt inch` (не `in`); `length * length` — P2301. В текст — `fmt_date(d, "contract")`.

ГЛОБАЛЬНЫЕ ИМЕНА: ЛЕНИВЫЕ, НЕИЗМЕНЯЕМЫЕ

```
total = price * qty      # выше определения — можно
price = 250.00
qty = 4
rate = 20 if vat else 0  # не: if vat: rate = 20
rows = [f(x) for x in xs]  # не: rows.append(…) в цикле
def build(): …           # или функция, возвращающая список
```

Определения (`x = …`, `a, b = …`, `def`, `|let`, параметры заголовка) собираются со всего файла, вычисляются при первом обращении и замораживаются; порядок в тексте не важен. Нечитаемое определение не вычисляется — и его ошибка не видна. Прочие операторы верхнего уровня (`for if` вызовы) — действия: выполняются после сбора, их имена локальны.

`x = 1` … `x = 2`	P2203; параметр заголовка в коде — тоже
`x += 1` верхнего уровня	P2105
`a = b + 1; b = a + 1`	P2204 цикл
`for r in xs: rows.append(r)`	P2318 заморожен
`for r in xs: total += r`	P2202 + P2501

ФУНКЦИИ

`def` с умолчаниями, `*args **kw`, `*, b`, рекурсия, вложенные функции и замыкания, `while`; внутри функции переменные изменяемы. Нет `lambda yield global nonlocal`. Умолчание замораживается: `def f(x, acc=[])` — P2318 при `append`; глобальный список, переданный в функцию, — тоже заморожен. Лимиты вместо `RecursionError`: глубина 500 (P2402), 20 с (P2406), 10^7 элементов.

МОДУЛИ И ФРАГМЕНТЫ

```
|import("lib/ціни")              ціни.з_пдв(x)  |ціни.macro(…)
|import("lib/ціни", "з_пдв")     з_пдв(x)
|import("lib/ціни", alias="ц")   ц.з_пдв(x)
# во фрагменте (.f.prml): имена закрыты, наружу — явно
@export                          # функция под своим именем
def знижка(сума, р): …
@export(name="g")                # под именем g
def _g(x): …
|export("підпис")  |let(x=1, export=True)  |export(y=2)
|include("f", ns="f")   # экспорт в f.*; rename={…}
```

Модуль: открыты все имена без `_`, экспорт не нужен; `модуль._имя` — ошибка. `|import` документа виден во фрагментах, `|import` фрагмента — только в нём; путь — от файла. `@export` — единственный декоратор: вне фрагмента P2610, над вложенной функцией P2106, в модуле — P2638.

ЗНАЧЕНИЯ В ТЕКСТЕ

Печати нет: только `|(x)`. Вставляются `str`, числа, даты, `content`; `None bool list dict record` — P2603 (не «None»): `|(x or "—")`, `join_list(xs)`, `str(x)`. Неоформленная дробь или дата — P2604. `markup("…") + "…"` — P2301: сначала строки, потом `markup()`.